Chương 7

TẮC NGHẼN (DEADLOCKS)

NỘI DUNG

1. **Mô hình hệ thống**
2. **Đặc điểm tắc nghẽn**
3. **Các phương pháp xử lý tắc nghẽn**
4. **Ngăn chặn tắc nghẽn**
5. **Tránh tắc nghẽn**
6. **Phát hiện tắc nghẽn**
7. **Khôi phục từ tắc nghẽn**

MỤC TIÊU

- ✓ Minh họa cách deadlock có thể xảy ra khi sử dụng khóa loại trừ
- ✓ Xác định bốn điều kiện đặc trưng gây ra tắc nghẽn
- ✓ Xác định tình huống tắc nghẽn trong đồ thị cấp phát tài nguyên
- ✓ Đánh giá bốn cách tiếp cận khác nhau để ngăn chặn tắc nghẽn
- ✓ Áp dụng thuật toán Banker để tránh tắc nghẽn
- ✓ Áp dụng thuật toán phát hiện tắc nghẽn
- ✓ Đánh giá các phương pháp phục hồi sau tắc nghẽn

1. Mô hình hệ thống

- Hệ thống bao gồm nhiều tài nguyên
- Ký hiệu tài nguyên: R_1, R_2, … , R_m
  - Chu kỳ CPU, không gian nhớ, thiết bị nhập/xuất
- Mỗi tài nguyên R_i có W_i thực thể (instance)
- Mỗi tiến trình sử dụng tài nguyên như sau:
  - Yêu cầu (request)
  - Sử dụng (use)
  - Giải phóng (release)

Khái niệm tắc nghẽn

- Tắc nghẽn (Deadlock): hai hoặc nhiều tiến trình đang chờ đợi vô hạn cho một sự kiện mà có thể được gây ra bởi chỉ một trong các tiến trình đang chờ
- Ví dụ:
  - **Cùng loại tài nguyên:** Xét hệ thống với 3 ổ đĩa CD RW. Giả sử có 3 tiến trình và mỗi tiến trình giữ một ổ đĩa. Nếu mỗi tiến trình yêu cầu thêm một ổ đĩa khác thì 3 tiến trình sẽ rơi vào trạng thái tắc nghẽn. Mỗi tiến trình đang chờ sự kiện "ổ CD RW được giải phóng"
  - **Khác loại tài nguyên:** Xét hệ thống có 1 máy in và 1 ổ đĩa DVD. Giả sử tiến trình P_i đang giữ ổ DVD và tiến trình P_j đang giữ máy in. nếu P_i yêu cầu máy in và P_j yêu cầu ổ DVD thì sẽ xảy ra tắc nghẽn

Tắc nghẽn trong ứng dụng đa tiểu trình

- Xem xét một chương trình với 2 khóa loại trừ được khởi tạo như sau:

```
pthread_mutex_t first_mutex;
pthread_mutex_t second_mutex;

pthread_mutex_init(&first_mutex,NULL);
pthread_mutex_init(&second_mutex,NULL);
```

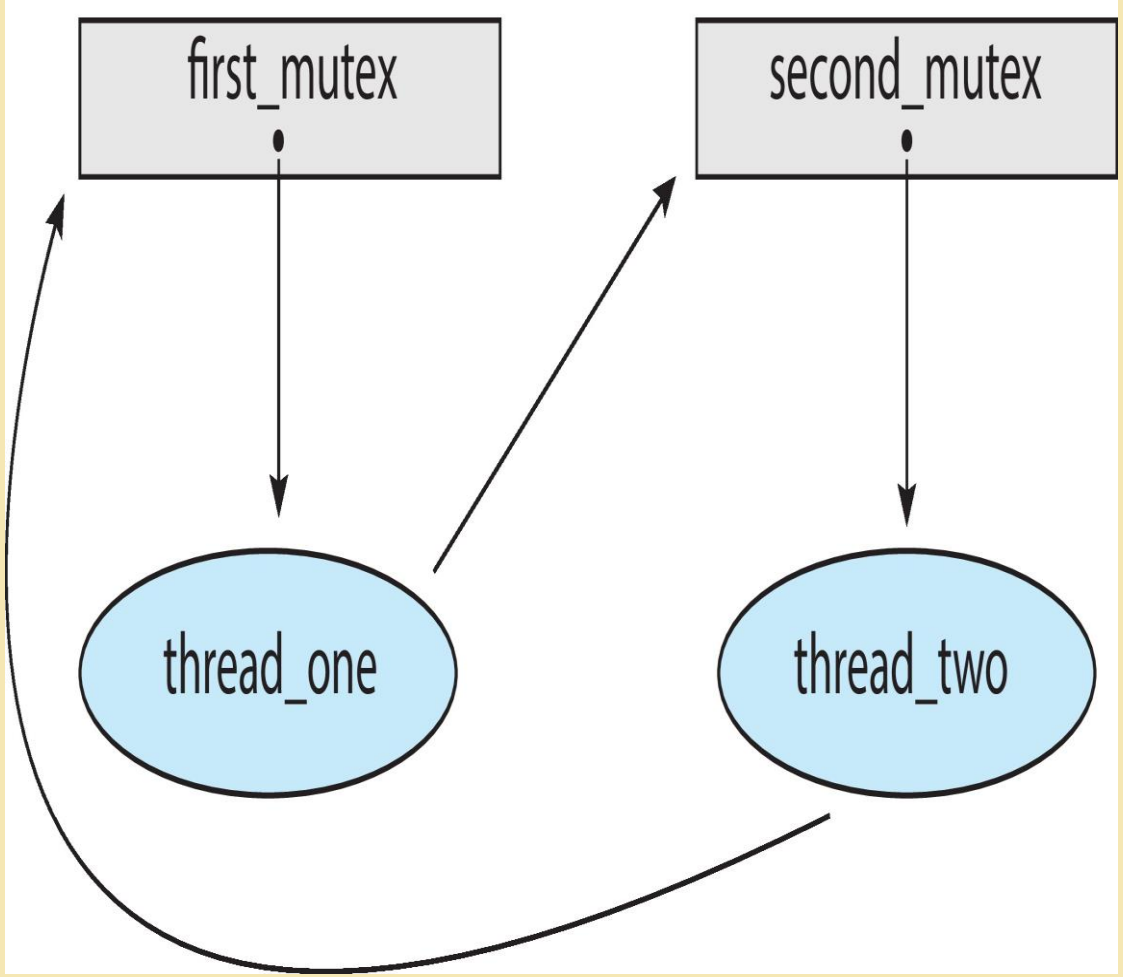
Tắc nghẽn trong ứng dụng đa tiểu trình

- Các tiểu trình thực thi công việc của nó

```
/* thread_one runs in this function */
void *do_work_one(void *param)
{
   pthread_mutex_lock(&first_mutex);
   pthread_mutex_lock(&second_mutex);
   /**
    * Do some work
    */
   pthread_mutex_unlock(&second_mutex);
   pthread_mutex_unlock(&first_mutex);

   pthread_exit(0);
}

/* thread_two runs in this function */
void *do_work_two(void *param)
{
   pthread_mutex_lock(&second_mutex);
   pthread_mutex_lock(&first_mutex);
   /**
    * Do some work
    */
   pthread_mutex_unlock(&first_mutex);
   pthread_mutex_unlock(&second_mutex);

   pthread_exit(0);
}
```

Tắc nghẽn trong ứng dụng đa tiểu trình

- Tắc nghẽn xảy ra nếu tiểu trình 1 có được ***first_mutex*** và tiểu trình 2 có được ***second_mutex***. Sau đó tiểu trình 1 đợi ***second_mutex*** và tiểu trình 2 đợi ***first_mutex***.
- Tình huống được minh họa bằng đồ thị cấp phát tài nguyên hình bên



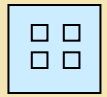
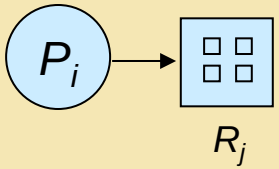
2. Đặc điểm tắc nghẽn

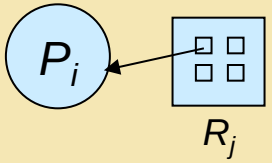
- Tắc nghẽn xuất hiện nếu 4 điều kiện sau xảy ra đồng thời
  - Độc quyền truy xuất (Mutual Exclusion): tại một thời điểm chỉ có duy nhất 1 tiến trình sử dụng tài nguyên
  - Giữ và đợi (Hold and Wait): một tiến trình giữ ít nhất 1 tài nguyên đang chờ thêm tài nguyên được giữ bởi tiến trình khác
  - Không ưu tiên (No preemption): một tài nguyên có thể chỉ được giải phóng một cách tự nguyện bởi một tiến trình đang giữ nó, sau khi tiến trình đó hoàn thành tác vụ của mình
  - Đợi vòng (Circular wait): tồn tại một tập tiến trình đang chờ $\{P_0, P_1, \ldots, P_n\}$. Trong đó, P_0 đang chờ tài nguyên được giữ bởi P_1, P_1 đang chờ tài nguyên được giữ bởi P_2, … , P_{n-1} đang chờ tài nguyên được giữ bởi P_n, P_n đang chờ tài nguyên được giữ bởi P_0.

Đồ thị cấp phát tài nguyên (RAG: Resource-Allocation Graph)

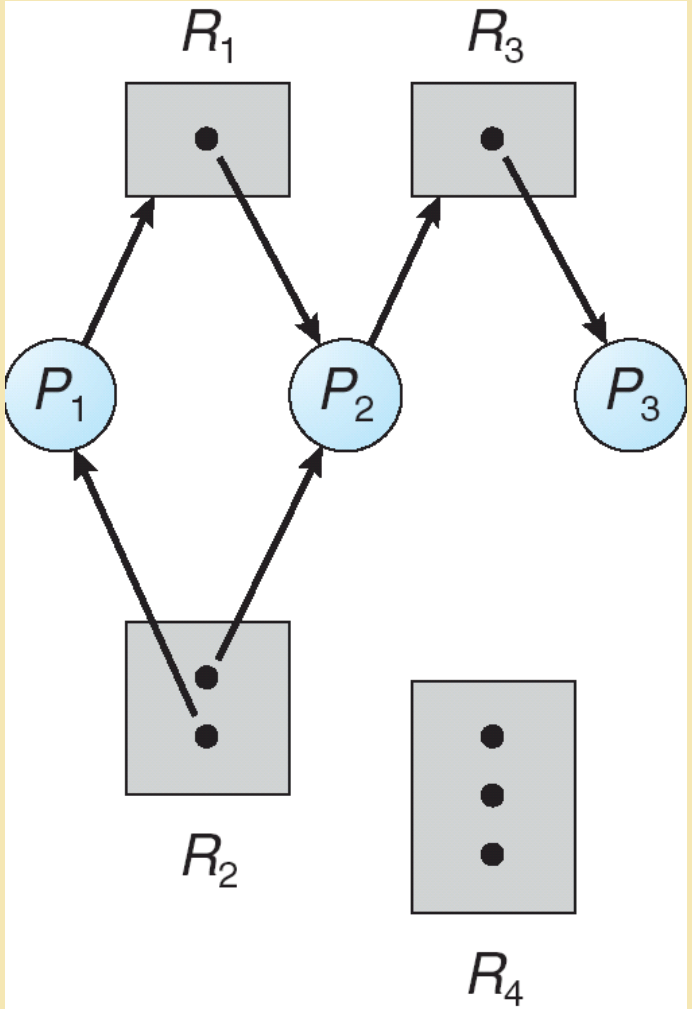
- Một tập hợp các đỉnh V và tập các cạnh E
- V được chia thành 2 loại
  - $P = \{P_1, P_2, \ldots, P_n\}$, tập tất cả các tiến trình trong hệ thống
  - $R = \{R_1, R_2, \ldots, R_m\}$, tập tất cả các tài nguyên của hệ thống
- Cạnh yêu cầu (**request edge**): cạnh có hướng từ $P_i \rightarrow R_j$
- Cạnh cấp phát (**assignment edge**): cạnh có hướng từ $R_j \rightarrow P_i$

Đồ thị cấp phát tài nguyên

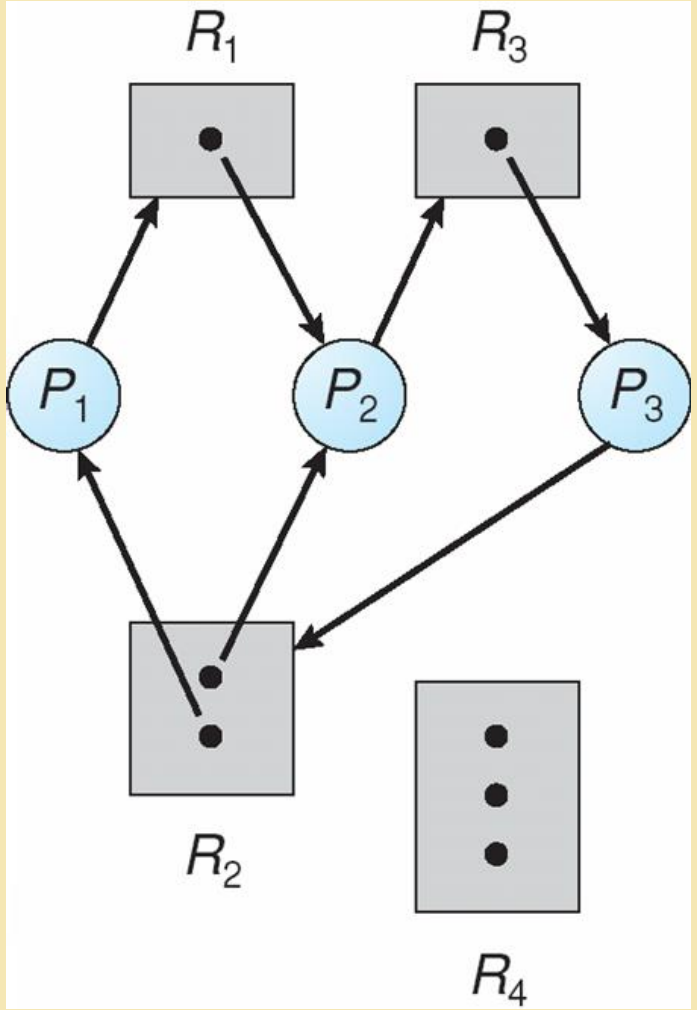
- Tiến trình
- Tài nguyên với 4 thực thể 
- P_i yêu cầu thực thể tài nguyên R_j 

- P_i đang giữ thực thể của R_j 

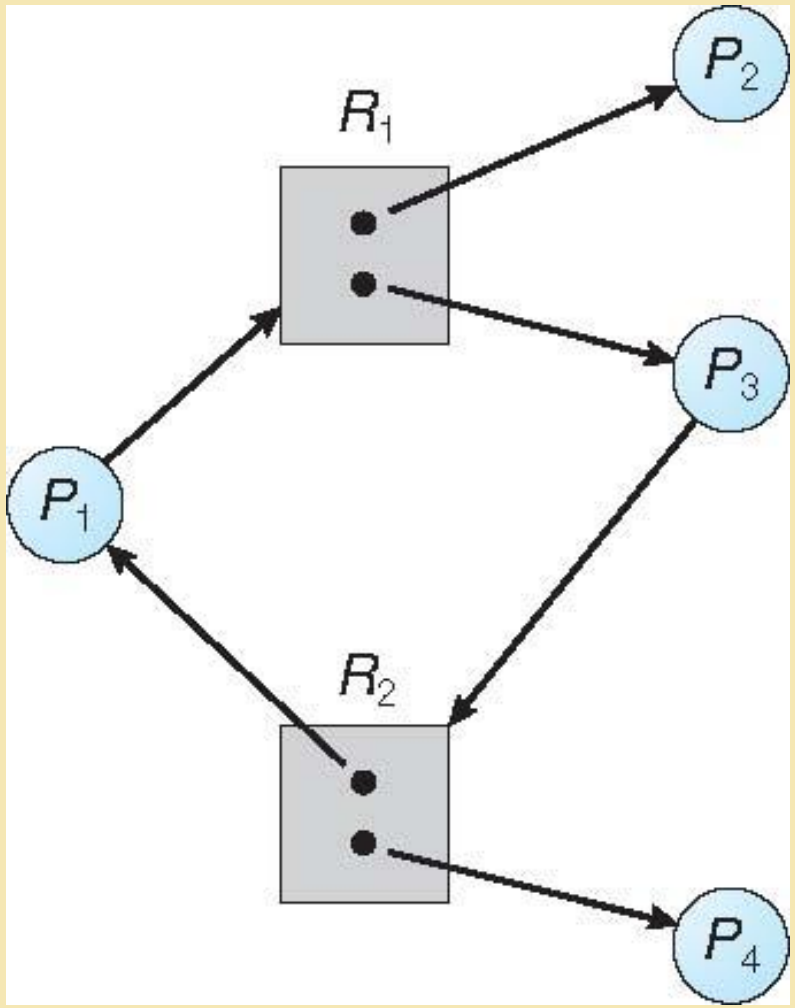
Ví dụ về RAG



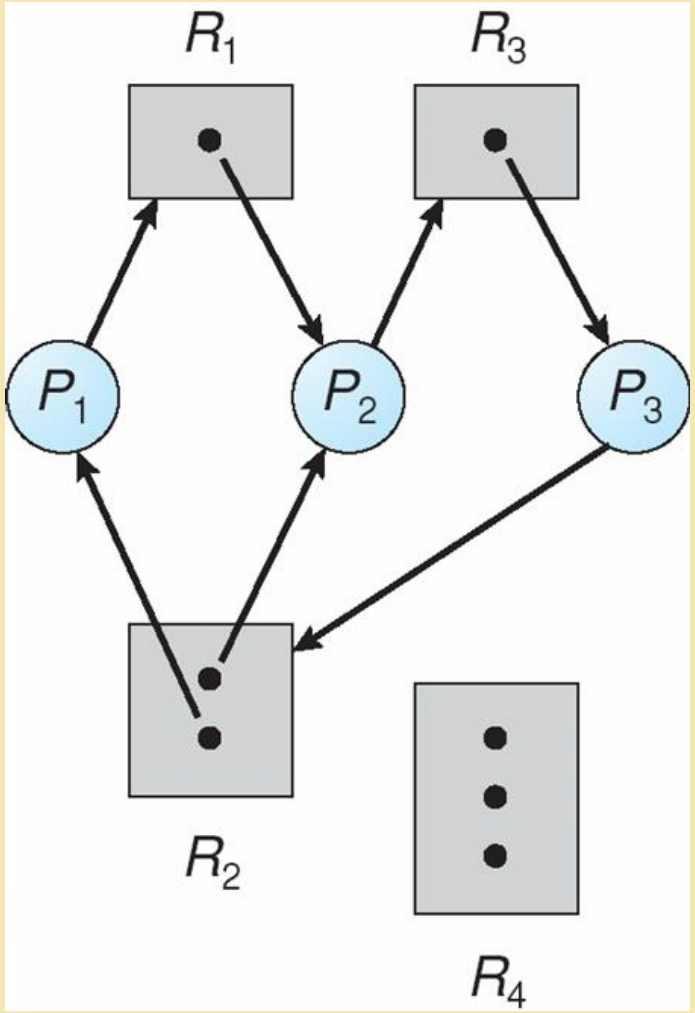
RAG có tắc nghẽn

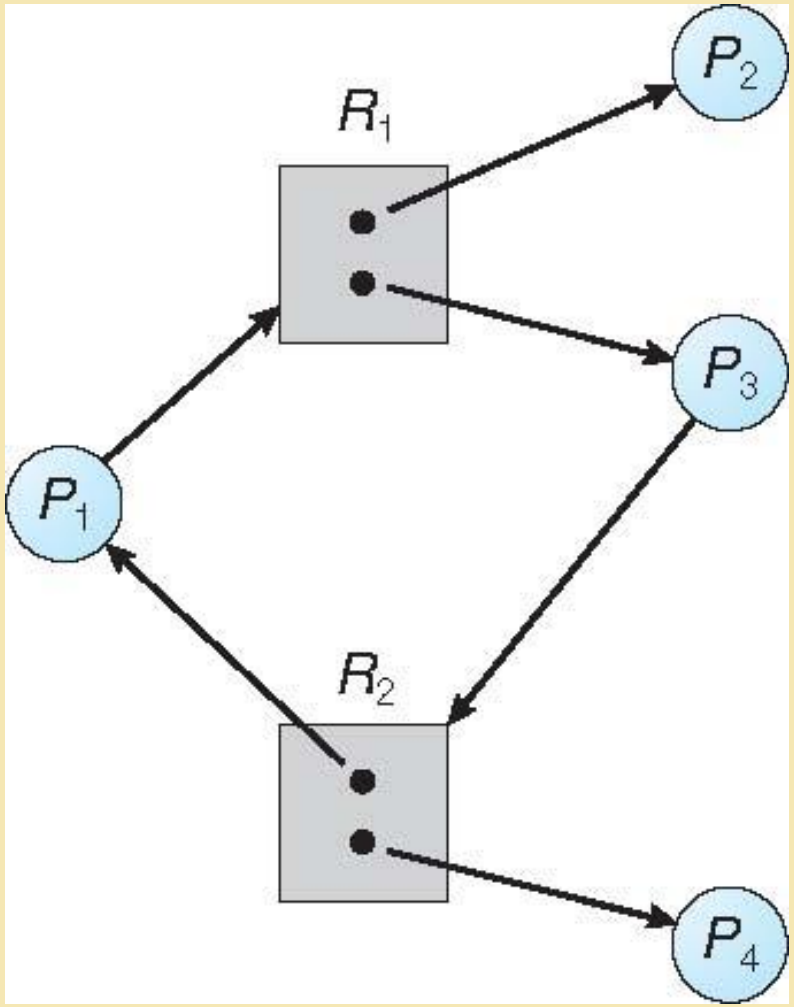


RAG có vòng nhưng không có tắc nghẽn



So sánh 2 RAG





Suy luận cơ bản

- Nếu đồ thị không chứa vòng → không tắc nghẽn
- Nếu đồ thị chứa vòng →
  - Nếu tài nguyên chỉ có 1 thực thể, thì tắc nghẽn
  - Nếu tài nguyên có nhiều thực thể, có khả năng tắc nghẽn

3. Các phương pháp xử lý tắc nghẽn

Các nhóm phương pháp bao gồm:

- Đảm bảo hệ thống sẽ không bao giờ rơi vào trạng thái tắc nghẽn:
  - Ngăn chặn tắc nghẽn
  - Tránh tắc nghẽn
- Cho phép hệ thống rơi vào trạng thái tắc nghẽn, sau đó khôi phục
- Bỏ qua việc xử lý tắc nghẽn, xem như hệ thống không bao giờ xảy ra tắc nghẽn.

4. Ngăn chặn tắc nghẽn

- Ngăn chặn 4 điều kiện làm xuất hiện tắc nghẽn
  - Độc quyền truy xuất – không được yêu cầu đối với tài nguyên có thể chia sẻ (vd: các tập tin chỉ đọc), phải nắm giữ đối với tài nguyên không thể chia sẻ
  - Giữ và đợi – phải đảm bảo rằng bất cứ khi nào tiến trình yêu cầu tài nguyên thì nó không được giữ bất kỳ tài nguyên nào khác
    - Đòi hỏi tiến trình yêu cầu và được cấp phát tất cả tài nguyên trước khi nó bắt đầu thực thi, hoặc cho phép tiến trình yêu cầu tài nguyên chỉ khi tiến trình không có tài nguyên được cấp phát
    - Việc sử dụng tài nguyên thấp, có thể *đói tài nguyên*

Ngăn chặn tắc nghẽn

- Không ưu tiên –
  - Nếu một tiến trình đang giữ một số tài nguyên và yêu cầu tài nguyên khác mà không được cấp phát ngay lập tức, thì tất cả tài nguyên hiện hành nó đang giữ được giải phóng
  - Tài nguyên bị chiếm được thêm vào danh sách tài nguyên mà các tiến trình đang chờ
  - Tiến trình sẽ được chạy lại chỉ khi nó chiếm lại tài nguyên cũ của nó, cũng như tài nguyên mới mà nó đang yêu cầu

Ngăn chặn tắc nghẽn

- Đợi vòng – đặt tất cả các loại tài nguyên vào thứ tự, và đòi hỏi mỗi tiến trình yêu cầu tài nguyên theo thứ tự tăng dần
  - Một hàm ánh xạ kiểu tài nguyên vào các thứ tự.
  - Ví dụ: F(tapedrive)=1; F(diskdrive)=5; F(printer)=12
  - Phương pháp ngăn chặn:
    - Bắt buộc mỗi tiến trình phải yêu cầu tài nguyên theo thứ tự tuyến tính tăng dần.
      - Ví dụ: chuỗi yêu cầu sau là hợp lệ: tapedrive, printer
      - Ví dụ: chuỗi yêu cầu là không hợp lệ: printer, tapedrive
    - Khi một tiến trình yêu cầu một tài nguyên R_j thì nó phải trả lại tất cả các tài nguyên R_i với $F(R_i)>F(R_j)$

5. Tránh tắc nghẽn

Yêu cầu hệ thống

- Mô hình đơn giản và hữu ích nhất là mỗi tiến trình sẽ khai báo số tài nguyên tối đa của mỗi loại mà tiến trình cần
- Các thuật toán tránh tắc nghẽn tự động kiểm tra trạng thái cấp phát tài nguyên để đảm bảo rằng không bao giờ xảy ra điều kiện đợi vòng
- Trạng thái cấp phát tài nguyên được xác định bằng số tài nguyên khả dụng và được cấp phát, và nhu cầu tối đa của các tiến trình

Trạng thái an toàn

- Khi một tiến trình yêu cầu một tài nguyên có sẵn, hệ thống phải quyết định cấp phát ngay lập tức nếu hệ thống đang ở trạng thái an toàn
- Hệ thống là trạng thái an toàn nếu tồn tại một chuỗi tiến trình $<P_1, P_2, \ldots, P_n>$ trong hệ thống mà mỗi P_i, tài nguyên mà P_i yêu cầu vẫn có thể được thỏa mãn bởi tài nguyên sẵn có hiện hành + tài nguyên được giữ bởi tất cả các P_j, với $j<i$
- Có nghĩa là:
  - Nếu tài nguyên P_i cần không có sẵn ngay lập tức thì P_i chờ cho đến khi tất cả P_j hoàn thành
  - Khi P_j hoàn thành, P_i có được tài nguyên cần, nó thực thi, trả tài nguyên lại và kết thúc
  - Khi P_i kết thúc, P_{i+1} có được tài nguyên cần của nó, và cứ tiếp tục

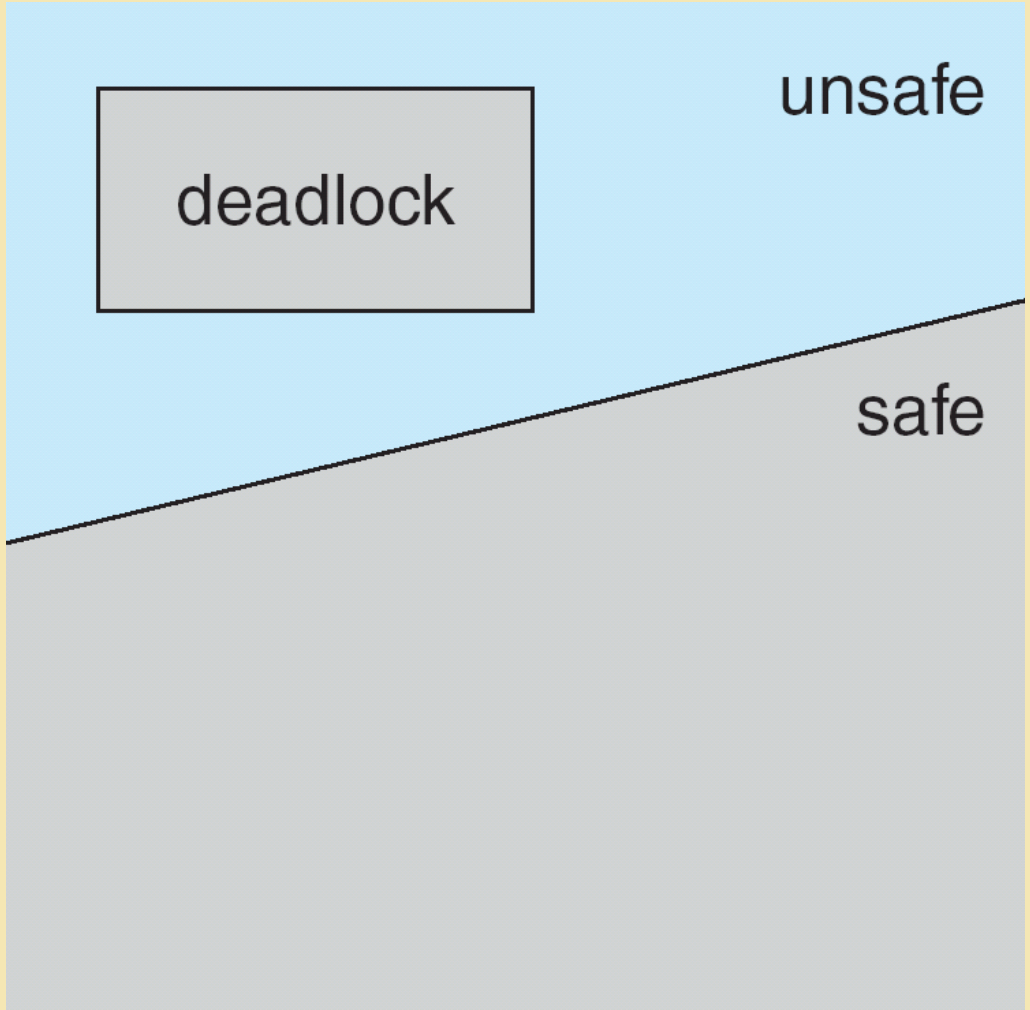
Ví dụ trạng thái an toàn

- Hệ thống có 12 ổ đĩa mềm và 3 tiến trình P0, P1, P2. Tiến trình P0 yêu cầu 10 ổ đĩa, P1 yêu cầu 4 và P2 yêu cầu 9.
- Tại thời điểm T0, P0 được cấp 5, P1 được cấp 2 và P2 được cấp 2, khi đó hệ thống còn 3 ổ đĩa.
- Ở thời điểm T0 hệ thống đang trong trạng thái an toàn với chuỗi an toàn là < P1, P0, P2 >
- Ở thời điểm T1, P2 yêu cầu thêm một ổ đĩa và được cấp phát, khi này hệ thống sẽ rơi vào trạng thái không an toàn.
- Lỗi là hệ thống đã cấp phát cho P2 thay vì buộc P2 phải đợi.

Suy luận cơ bản

- Nếu hệ thống ở trạng thái an toàn $\Rightarrow$ không có tắc nghẽn
- Nếu hệ thống ở trạng thái không an toàn $\Rightarrow$ có khả năng tắc nghẽn
- Tránh tắc nghẽn $\Rightarrow$ đảm bảo rằng hệ thống không bao giờ rơi vào trạng thái không an toàn

Trạng thái: an toàn, không an toàn, tắc nghẽn



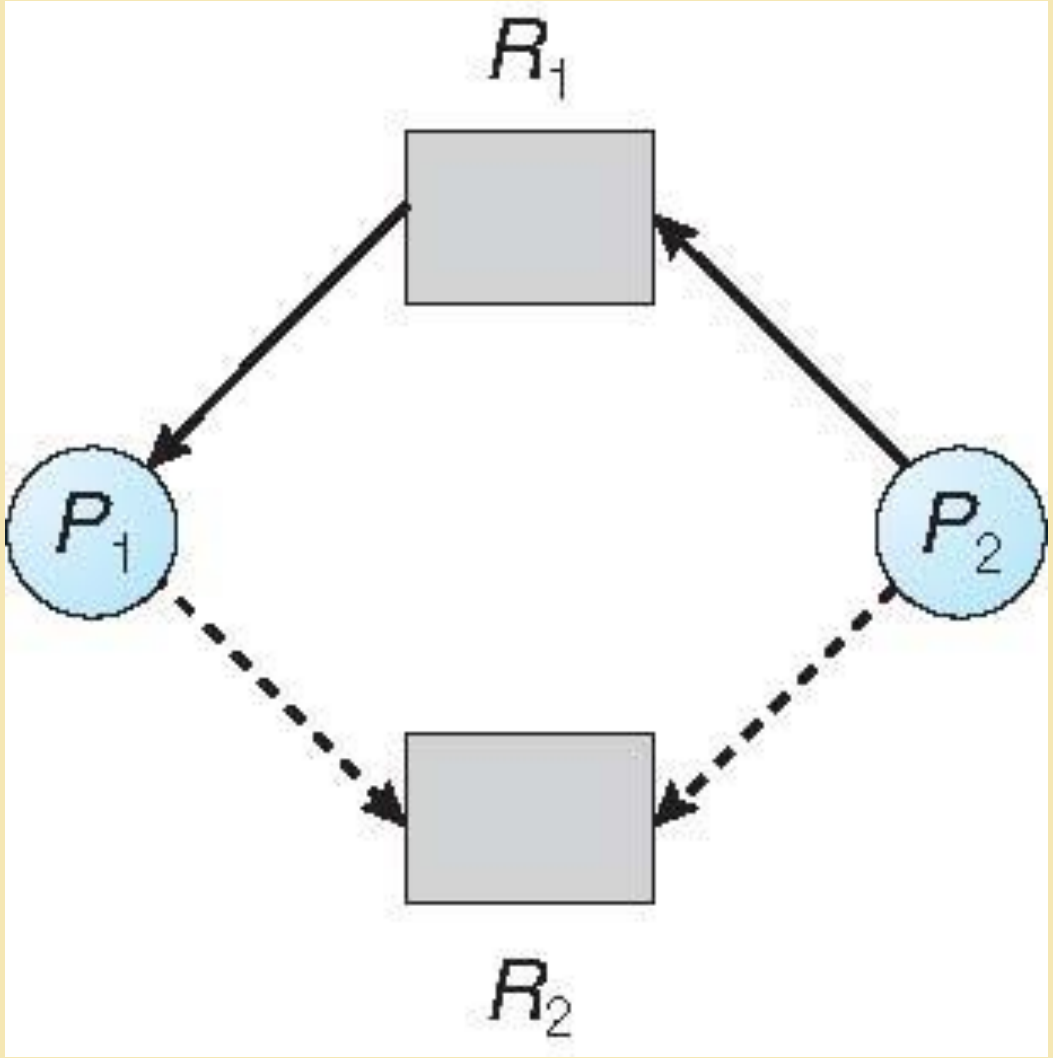
Các giải thuật tránh tắc nghẽn

- Tài nguyên có một thực thể
  - Sử dụng đồ thị cấp phát tài nguyên (RAG)
- Tài nguyên có nhiều thực thể
  - Sử dụng giải thuật Banker

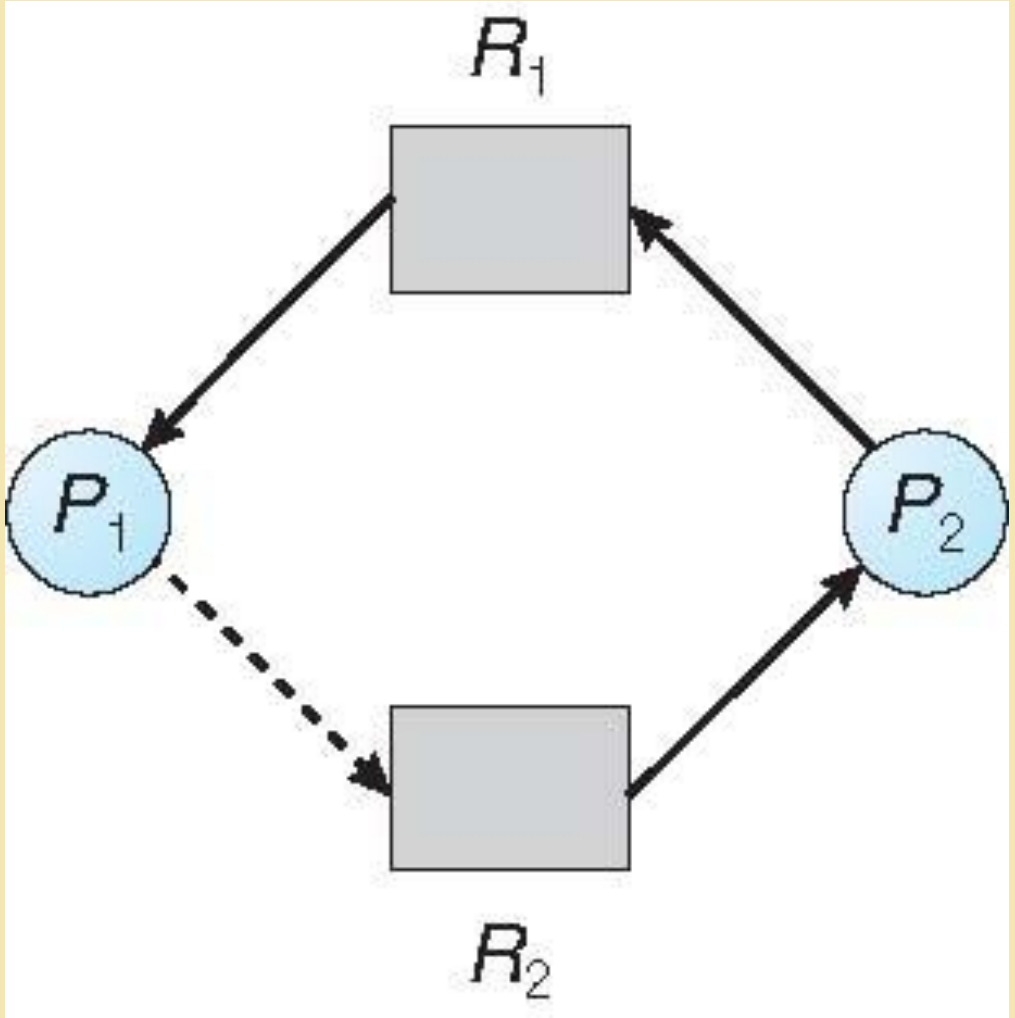
Phương án RAG

- Cạnh đòi hỏi (**claim edge**) $P_i \rightarrow R_j$ là cạnh cho biết tiến trình P_i có thể yêu cầu tài nguyên R_j, nó được biểu diễn bằng 1 đường nét đứt.
- Cạnh đòi hỏi (claim edge) chuyển thành cạnh yêu cầu (request edge) khi một tiến trình yêu cầu tài nguyên.
- Cạnh yêu cầu được chuyển thành cạnh cấp phát (assignment edge) khi tài nguyên được cấp cho tiến trình.
- Khi tài nguyên được giải phóng bởi tiến trình, cạnh cấp phát chuyển lại thành cạnh đòi hỏi.
- Các tài nguyên phải được gán 1 độ ưu tiên trong hệ thống.

Phương án RAG



Trạng thái không an toàn trong RAG



Giải thuật RAG

- Giả sử tiến trình P_i yêu cầu tài nguyên R_j
- Yêu cầu được cấp phát chỉ khi việc chuyển từ cạnh yêu cầu thành cạnh cấp phát không tạo thành vòng trong đồ thị.

Giải thuật Banker

- Áp dụng cho tài nguyên có nhiều thực thể
- Mỗi tiến trình phải sử dụng độ ưu tiên cao nhất
- Khi một tiến trình yêu cầu tài nguyên, nó có thể phải chờ
- Khi một tiến trình có đầy đủ tài nguyên, nó phải trả lại chúng trong thời gian xác định

Cấu trúc dữ liệu của giải thuật Banker

Gọi ***n*** là số tiến trình; ***m*** là số tài nguyên

- **Available**: vector với độ dài m, là số thực thể đang khả dụng (có sẵn). Nếu available[j]=k, có k thực thể của tài nguyên Rj là khả dụng.
- **Max**: ma trận nxm, số thực thể tối đa mà tiến trình yêu cầu. Nếu Max[i,j]=k, thì tiến trình Pi có thể yêu cầu nhiều nhất k thực thể của tài nguyên Rj
- **Allocation**: ma trận nxm, số thực thể của tài nguyên đã cấp phát cho tiến trình. Nếu Allocation[i,j]=k, thì hiện hành tiến trình Pi được cấp phát k thực thể của tài nguyên Rj
- **Need**: ma trận nxm, số thực thể tiến trình cần để hoàn thành tác vụ. Nếu Need[i,j]=k, thì tiến trình Pi cần thêm k thực thể của Rj để hoàn thành tác vụ

Need[i,j]=Max[i,j]-Allocation[i,j]

Giải thuật xác định trạng thái an toàn

1. Gọi ***Work*** và ***Finish*** là 2 vector với độ dài tương ứng là ***m*** và ***n***. Khởi tạo:

   Work* = *Available

   Finish* [*i*] = *false với ***i* = 0, 1, ..., *n*- 1**

2. Tìm ***i*** thỏa 2 điều kiện:

   a) ***Finish[i]=false***

   b) $\boldsymbol{Need_i \leq Work}$

   Nếu không tồn tại ***i*** như vậy thì qua Bước 4

3. $\boldsymbol{Work = Work + Allocation_i}$

   Finish[i] = true

   Quay lại Bước 2

4. Nếu ***Finish* [*i*] == *true*** với tất cả ***i***, thì hệ thống ở trạng thái an toàn

Ví dụ về giải thuật Banker

- 5 tiến trình: P_0 đến P_4
- 3 loại tài nguyên:
  - A (10 thực thể); B (5 thực thể) và C (7 thực thể)
- Hiện trạng tại thời điểm T_0:

	Allocation	***Max***	***Available***
	A B C	*A B C*	*A B C*
P_0	0 1 0	7 5 3	3 3 2
P_1	2 0 0	3 2 2	
P_2	3 0 2	9 0 2	
P_3	2 1 1	2 2 2	
P_4	0 0 2	4 3 3	

Ví dụ về giải thuật Banker

- Ma trận ***Need*** được xác định bởi ***Max – Allocation***

	Need
	A B C
P_0	7 4 3
P_1	1 2 2
P_2	6 0 0
P_3	0 1 1
P_4	4 3 1

	Allocation	***Max***	***Available***
	A B C	*A B C*	*A B C*
P_0	0 1 0	7 5 3	3 3 2
P_1	2 0 0	3 2 2	
P_2	3 0 2	9 0 2	
P_3	2 1 1	2 2 2	
P_4	0 0 2	4 3 3	

- Hệ thống ở trạng thái an toàn vì chuỗi ***<P_1, P_3, P_4, P_2, P_0>*** thỏa mãn điều kiện an toàn

Giải thuật yêu cầu tài nguyên của tiến trình P_i

Request$_i$ – vector tài nguyên yêu cầu của tiến trình ***P$_i$***. Nếu ***Request$_i$[j]=k*** thì tiến trình ***P$_i$*** muốn ***k*** thực thể của tài nguyên ***R$_j$***

1. Nếu ***Request$_i$*** ≤ ***Need$_i$*** tới Bước 2. Ngược lại, đưa ra lỗi vì tiến trình vượt quá đòi hỏi tối đa
2. Nếu ***Request$_i$*** ≤ ***Available*** tới Bước 3. Ngược lại, Pi phải chờ vì tài nguyên không có sẵn
3. Giả sử hệ thống đã cấp phát cho Pi các tài nguyên mà nó yêu cầu và cập nhật tình trạng hệ thống như sau:

   Available = Available – Request$_i$;
   Allocation$_i$ = Allocation$_i$ + Request$_i$;
   Need$_i$ = Need$_i$ – Request$_i$;

- Nếu trạng thái là an toàn ⇒ tài nguyên được cấp phát cho ***P$_i$***
- Nếu trạng thái là không an toàn ⇒ ***P$_i$*** phải chờ, và trạng thái cấp phát tài nguyên cũ được phục hồi

Ví dụ về giải thuật Banker: **P1 yêu cầu (1,0,2)**

- Kiểm tra $Request_1 \leq Available$ $((1,0,2) \leq (3,3,2) \Rightarrow$ đúng)

	Allocation	*Need*	*Available*
	A B C	*A B C*	*A B C*
P_0	0 1 0	7 4 3	2 3 0
P_1	3 0 2	0 2 0	
P_2	3 0 2	6 0 0	
P_3	2 1 1	0 1 1	
P_4	0 0 2	4 3 1	

	Allocation	***Max***	***Available***
	A B C	*A B C*	*A B C*
P_0	0 1 0	7 5 3	3 3 2
P_1	2 0 0	3 2 2	
P_2	3 0 2	9 0 2	
P_3	2 1 1	2 2 2	
P_4	0 0 2	4 3 3	

- Thực hiện giải thuật xác định trạng thái an toàn, thấy rằng chuỗi $<P_1, P_3, P_4, P_0, P_2>$ đáp ứng yêu cầu an toàn
- Yêu cầu (3,3,0) bởi P_4 có thể được cấp phát không?
- Yêu cầu (0,2,0) bởi P_0 có thể được cấp phát không?

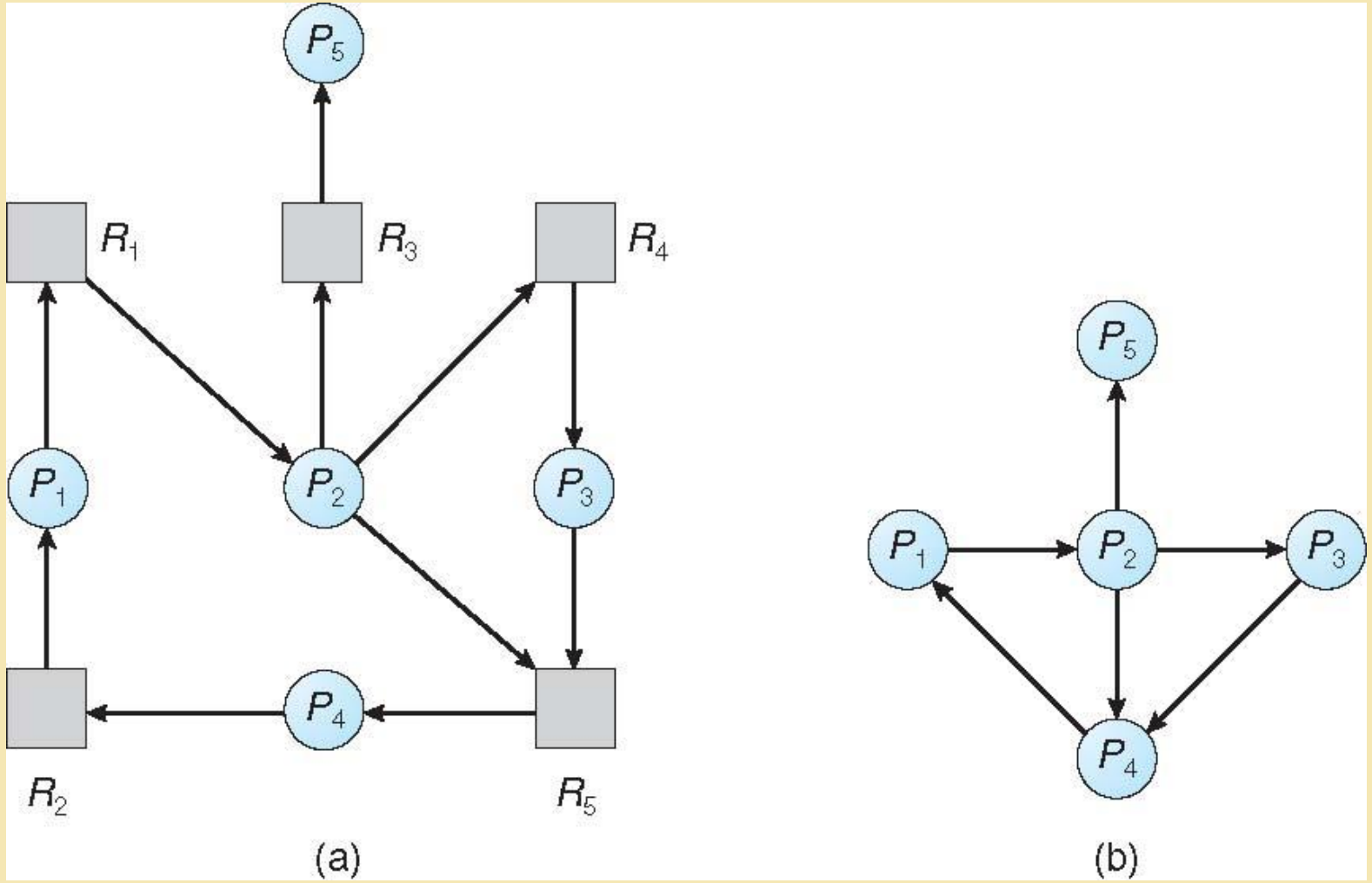
6. Phát hiện tắc nghẽn

- Cho phép hệ thống rơi vào trạng thái tắc nghẽn
- Giải thuật phát hiện tắc nghẽn
- Cách thức phục hồi

Tài nguyên có một thực thể

- Đồ thị chờ (wait-for graph)
  - Đỉnh là các tiến trình
  - $P_i \rightarrow P_j$ nếu P_i đang chờ P_j
- Gọi một giải thuật tìm kiếm một chu trình trong đồ thị. Nếu có một chu trình thì tồn tại tắc nghẽn
- Giải thuật để phát hiện chu trình trong đồ thị đòi hỏi n^2 thao tác lệnh, trong đó n là số đỉnh của đồ thị

Đồ thị cấp phát tài nguyên và đồ thị chờ



Tài nguyên có nhiều thực thể

- **Available**: vector với độ dài m, cho biết số tài nguyên khả dụng của mỗi loại
- **Allocation**: ma trận nxm, xác định số tài nguyên của mỗi loại được cấp phát hiện hành cho mỗi tiến trình
- **Request**: ma trận nxm, cho biết yêu cầu hiện tại của mỗi tiến trình. Nếu Request[i][j]=k thì tiến trình P_i đang yêu cầu thêm k thực thể của tài nguyên R_j

Giải thuật phát hiện tắc nghẽn

1. Gọi **Work** và **Finish** là các vector với độ dài tương ứng là m và n, được khởi tạo như sau:
   a) ***Work = available***
   b) Với mỗi i=1,2,…,n nếu ***Allocation***$_i \neq$ **0** thì ***Finish*[i] *= false***; ngược lại, ***Finish*[i] = *true***
2. Tìm i sao cho thỏa 2 điều kiện:
   a) ***Finish[i] == false***
   b) ***Request***$_i \leq$ ***Work***

   Nếu i không tồn tại thì qua Bước 4
3. ***Work = Work + Allocation***$_i$
   Finish[i] = true
   Quay lại Bước 2
4. Nếu ***Finish[i] == false*** với một vài i, $1 \leq i \leq n$, thì hệ thống trong trạng thái tắc nghẽn. Ngoài ra, nếu ***Finish*[*i*] == *false*** thì P_i bị tắc nghẽn

Ví dụ về giải thuật phát hiện tắc nghẽn

- Cho 5 tiến trình từ P0 đến P4; 3 tài nguyên: A (7 thực thể), B (2 thực thể) và C (6 thực thể)
- Hiện trạng tại thời điểm T0

	Allocation	*Request*	*Available*
	A B C	*A B C*	*A B C*
P_0	0 1 0	0 0 0	0 0 0
P_1	2 0 0	2 0 2	
P_2	3 0 3	0 0 0	
P_3	2 1 1	1 0 0	
P_4	0 0 2	0 0 2	

- Chuỗi <P_0, P_2, P_3, P_1, P_4> sẽ cho kết quả ***Finish[i] = true*** với mọi i

Ví dụ về giải thuật phát hiện tắc nghẽn

- P_2 yêu cầu thêm 1 thực thể tài nguyên C

	Request
	A B C
P_0	0 0 0
P_1	2 0 2
P_2	0 0 1
P_3	1 0 0
P_4	0 0 2

	Allocation	*Request*	*Available*
	A B C	*A B C*	*A B C*
P_0	0 1 0	0 0 0	0 0 0
P_1	2 0 0	2 0 2	
P_2	3 0 3	0 0 0	
P_3	2 1 1	1 0 0	
P_4	0 0 2	0 0 2	

- Hệ thống có an toàn?
  - Có thể đòi lại tài nguyên được giữ bởi tiến trình P0, nhưng không đủ tài nguyên để thi hành các tiến trình khác
  - Tắc nghẽn tồn tại, bao gồm các tiến trình P_1, P_2, P_3 và P_4

Sử dụng giải thuật phát hiện tắc nghẽn

- Khi nào chúng ta nên gọi giải thuật? Phụ thuộc vào 2 nhân tố:
  - Tắc nghẽn có thường xảy ra hay không?
  - Khi tắc nghẽn xảy ra, có bao nhiêu tiến trình bị ảnh hưởng?
- Nếu giải thuật phát hiện được gọi một cách tùy tiện, có thể có nhiều chu trình trong đồ thị tài nguyên và vì vậy chúng ta sẽ không thể nói: nhiều tiến trình bị tắc nghẽn "gây ra" sự tắc nghẽn.

7. Khôi phục từ tắc nghẽn: kết thúc tiến trình

- Hủy bỏ tất cả các tiến trình bị tắc nghẽn
- Hủy bỏ một tiến trình tại một thời điểm cho đến khi chu kỳ tắc nghẽn được loại bỏ
- Chúng ta nên chọn thứ tự nào để hủy bỏ?
  1. Độ ưu tiên của tiến trình
  2. Tiến trình tính toán bao lâu và thời gian hoàn thành lâu bao nhiêu
  3. Tài nguyên tiến trình đã sử dụng
  4. Tài nguyên tiến trình cần để hoàn thành
  5. Bao nhiêu tiến trình sẽ cần kết thúc
  6. Tiến trình là tương tác hay theo lô

Khôi phục từ tắc nghẽn: sự ưu tiên tài nguyên

- Lựa chọn tiến trình bị thu hồi tài nguyên (tiến trình nạn nhân) – chi phí thấp nhất
- Rollback – quay trở lại một số trạng thái an toàn, khởi động lại tiến trình ở trạng thái đó
- Starvation – cùng một tiến trình có thể luôn bị chọn là nạn nhân

Thắc mắc & Giải đáp